

Transfer Dependency Reuse: Inferring N 's Perturbation Dependencies from N_{pre} 's via Weight Differences

1 Background

We briefly recall the dependency analysis from VHAGaR and how **BRIDGE** uses it.

Dependency analysis. Given a network N with weights W_m at layer m , a perturbation type P with bound ε , and the perturbation set $\mathcal{I}_\varepsilon(x)$, the dependency analysis computes a *dependency vector* $\varphi_m \in \{0, +1, -1, \text{NaN}\}^{k_m}$ at each layer m , where k_m is the number of neurons in layer m . The dependency $\varphi_{m,k}$ characterizes the relationship between the clean activation $z_{m,k}^{\text{org}}$ (from input x) and the perturbed activation $z_{m,k}^{\text{pert}}$ (from input $x' \in \mathcal{I}_\varepsilon(x)$):

- $\varphi_{m,k} = 0$: equal — $z_{m,k}^{\text{pert}} = z_{m,k}^{\text{org}}$ for all x, x' .
- $\varphi_{m,k} = +1$: monotonically increasing — $z_{m,k}^{\text{pert}} \geq z_{m,k}^{\text{org}}$ for all x, x' .
- $\varphi_{m,k} = -1$: monotonically decreasing — $z_{m,k}^{\text{pert}} \leq z_{m,k}^{\text{org}}$ for all x, x' .
- $\varphi_{m,k} = \text{NaN}$: unknown — no monotonicity can be established analytically.

For each neuron with $\varphi_{m,k} \in \{0, +1, -1\}$, the analysis adds the corresponding ordering constraint to the MIP. These constraints are *tightening*: they prune the feasible region but are not required for the correctness of the MIP encoding.

Dependency propagation. The dependency vector is computed inductively. The base case φ_0 is determined by the perturbation type (e.g., for brightness with $\varepsilon > 0$: $\varphi_{0,k} = +1$ for all pixels). For a linear layer with weight matrix W_m , the propagation rule computes, for each output neuron k :

$$o_k = \sum_i W_m[k, i] \cdot \varphi_{m-1, i}, \quad t_k = \sum_i |W_m[k, i]| \cdot |\varphi_{m-1, i}|, \quad (1)$$

where the sums range over inputs i with $\varphi_{m-1, i} \neq 0$ and $\varphi_{m-1, i} \neq \text{NaN}$. The ratio $r_k = o_k/t_k$ determines the dependency:

$$\varphi_{m,k} = \begin{cases} +1 & \text{if } r_k = +1, \\ -1 & \text{if } r_k = -1, \\ 0 & \text{if } t_k = 0 \text{ and } o_k = 0, \\ \text{NaN} & \text{otherwise (i.e., } |r_k| < 1). \end{cases} \quad (2)$$

Inputs i with $\varphi_{m-1,i} = \text{NaN}$ poison the computation: if any such input has $W_m[k, i] \neq 0$, then $\varphi_{m,k} = \text{NaN}$. For neurons with $\varphi_{m,k} = \text{NaN}$, VHAGaR optionally solves LP sub-problems to determine whether the ordering holds given the MIP bounds. These LP sub-problems are the expensive part of the dependency analysis.

Key observation. The ratio $r_k = \pm 1$ if and only if *all* non-zero terms $W_m[k, i] \cdot \varphi_{m-1,i}$ share the same sign. Equivalently:

$$\varphi_{m,k} \in \{+1, -1\} \iff \forall i \text{ with } \varphi_{m-1,i} \notin \{0, \text{NaN}\} : \text{sgn}(W_m[k, i]) \cdot \text{sgn}(\varphi_{m-1,i}) \text{ is constant.} \quad (3)$$

This means the dependency depends only on the *signs* of the weights on relevant paths, not on their magnitudes.

2 Transfer Dependency Reuse

Setting. We are in BRIDGE’s transfer setting. We have a pretrained network N_{pre} with weights W_m^{pre} and a target network N with weights $W_m^N = W_m^{\text{pre}} + \Delta W_m$. Standard mode has already computed the dependency vectors φ_m^{pre} for N_{pre} . We wish to compute the dependency vectors φ_m^N for N *without solving any LP sub-problems*, by leveraging φ_m^{pre} and the weight differences ΔW_m .

Goal. For each neuron (m, k) , either:

1. Confirm that $\varphi_{m,k}^N = \varphi_{m,k}^{\text{pre}}$ (reuse the constraint), or
2. Conservatively set $\varphi_{m,k}^N = \text{NaN}$ (add no constraint for this neuron).

Since dependency constraints are tightening, option (2) preserves soundness.

2.1 Sign-Flip Condition

Definition 1 (Sign flip). Weight $W_m[k, i]$ has a *sign flip* between N_{pre} and N if

$$\text{sgn}(W_m^N[k, i]) \neq \text{sgn}(W_m^{\text{pre}}[k, i]),$$

where $\text{sgn}(0) = 0$. We define the sign-flip set at layer m as:

$$F_m = \{(k, i) : \text{sgn}(W_m^N[k, i]) \neq \text{sgn}(W_m^{\text{pre}}[k, i])\}.$$

Note that ΔW_m alone is not sufficient to detect a sign flip — we need $\text{sgn}(W_m^{\text{pre}}[k, i])$ and $\text{sgn}(W_m^{\text{pre}}[k, i] + \Delta W_m[k, i])$. A sign flip occurs when $|\Delta W_m[k, i]| > |W_m^{\text{pre}}[k, i]|$ and the signs differ, or when a zero weight becomes non-zero (or vice versa).

Definition 2 (Relevant inputs). For neuron k at layer m , the *relevant input set* with respect to dependency vector φ_{m-1} is:

$$R_{m,k}(\varphi_{m-1}) = \{i : \varphi_{m-1,i} \notin \{0, \text{NaN}\} \text{ and } W_m[k, i] \neq 0\}.$$

2.2 Layer-by-Layer Reuse Algorithm

We propagate the inferred dependency vector $\tilde{\varphi}_m^N$ layer by layer. The key insight is that at each layer, if $\tilde{\varphi}_{m-1}^N = \varphi_{m-1}^{\text{pre}}$ for all inputs relevant to neuron k , and no weight feeding neuron k has a sign flip, then $\varphi_{m,k}^N = \varphi_{m,k}^{\text{pre}}$.

Algorithm 1: TransferDependencyReuse($N_{\text{pre}}, N, P, \varepsilon, \{\varphi_m^{\text{pre}}\}_{m=0}^L$)

Input: Networks N_{pre}, N with weights W_m^{pre}, W_m^N ; perturbation P with bound ε ; precomputed dependency vectors $\{\varphi_m^{\text{pre}}\}_{m=0}^L$

Output: Inferred dependency vectors $\{\tilde{\varphi}_m^N\}_{m=0}^L$

```

1  $\tilde{\varphi}_0^N \leftarrow \varphi_0^{\text{pre}}$  // Same perturbation  $\Rightarrow$  same input deps
2 for  $m = 1, \dots, L$  do
3   Compute  $F_m = \{(k, i) : \text{sgn}(W_m^N[k, i]) \neq \text{sgn}(W_m^{\text{pre}}[k, i])\}$ 
4   for each neuron  $k \in \{1, \dots, k_m\}$  do
5      $\text{poisoned} \leftarrow \exists i \text{ s.t. } W_m^N[k, i] \neq 0 \text{ and } \tilde{\varphi}_{m-1,i}^N = \text{NaN}$  // NaN input?
6      $\text{diverged} \leftarrow \exists i \in R_{m,k}(\varphi_{m-1}^{\text{pre}}) \text{ s.t. } \tilde{\varphi}_{m-1,i}^N \neq \varphi_{m-1,i}^{\text{pre}}$  // Input deps changed?
7      $\text{flipped} \leftarrow \exists i \in R_{m,k}(\tilde{\varphi}_{m-1}^N) \text{ s.t. } (k, i) \in F_m$  // Weight sign flip?
8     if  $\text{poisoned}$  or  $\text{diverged}$  or  $\text{flipped}$  then
9        $\tilde{\varphi}_{m,k}^N \leftarrow \text{NaN}$  // Cannot confirm  $\Rightarrow$  skip
10    else
11       $\tilde{\varphi}_{m,k}^N \leftarrow \varphi_{m,k}^{\text{pre}}$  // Safe to reuse
12 return  $\{\tilde{\varphi}_m^N\}_{m=0}^L$ 

```

Constraint encoding. After Algorithm 1, for each neuron (m, k) with $\tilde{\varphi}_{m,k}^N \in \{0, +1, -1\}$, we add the corresponding constraint to the transfer MIP:

$$\begin{aligned}
\tilde{\varphi}_{m,k}^N = 0 &\implies z_{m,k}^{\text{pert}} = z_{m,k}^{\text{org}}, \\
\tilde{\varphi}_{m,k}^N = +1 &\implies z_{m,k}^{\text{pert}} \geq z_{m,k}^{\text{org}}, \\
\tilde{\varphi}_{m,k}^N = -1 &\implies z_{m,k}^{\text{pert}} \leq z_{m,k}^{\text{org}}.
\end{aligned}$$

Neurons with $\tilde{\varphi}_{m,k}^N = \text{NaN}$ receive no constraint.

3 Soundness Proof

We prove that the algorithm is sound in two steps: (1) every reused dependency is correct, and (2) skipping a dependency preserves soundness of δ_{diff} .

3.1 Correctness of Reused Dependencies

Lemma 3 (Same-sign preservation). *Let φ_{m-1} be a dependency vector, and let W, W' be two weight matrices for layer m such that for all $i \in R_{m,k}(\varphi_{m-1})$:*

$$\text{sgn}(W[k, i]) = \text{sgn}(W'[k, i]).$$

Then the dependency propagation (2) yields the same result for neuron k under both W and W' .

Proof. Recall from (3) that $\varphi_{m,k} \in \{+1, -1\}$ if and only if all non-zero terms $W[k, i] \cdot \varphi_{m-1,i}$ share the same sign. Since $\text{sgn}(W[k, i]) = \text{sgn}(W'[k, i])$ for all relevant i , the sign of each term $W[k, i] \cdot \varphi_{m-1,i}$ equals the sign of $W'[k, i] \cdot \varphi_{m-1,i}$. Therefore:

- If all terms share the same sign under W , they also share the same sign under W' , and both yield the same $\varphi_{m,k} \in \{+1, -1\}$.
- If $t_k = 0$ under W (no relevant inputs), then $t_k = 0$ under W' as well, so both yield $\varphi_{m,k} = 0$.

In both cases, the propagation result is identical. $\square$

Theorem 4 (Correctness of reused dependencies). *For every neuron (m, k) where Algorithm 1 sets $\tilde{\varphi}_{m,k}^N = \varphi_{m,k}^{\text{pre}}$, the true dependency of N satisfies $\varphi_{m,k}^N = \varphi_{m,k}^{\text{pre}}$. That is, the reused constraint is a valid ordering constraint for N .*

Proof. By induction on the layer index m .

Base case ($m = 0$): The input-layer dependency φ_0 is determined solely by the perturbation type P and bound ε , which are the same for both networks. Therefore $\varphi_0^N = \varphi_0^{\text{pre}} = \tilde{\varphi}_0^N$.

Inductive step: Assume that for all neurons i at layer $m-1$, if $\tilde{\varphi}_{m-1,i}^N \neq \text{NaN}$ then $\tilde{\varphi}_{m-1,i}^N = \varphi_{m-1,i}^N$ (i.e., every non-NaN inferred dependency at the previous layer is correct).

Consider neuron k at layer m where the algorithm sets $\tilde{\varphi}_{m,k}^N = \varphi_{m,k}^{\text{pre}}$. By the algorithm's conditions, the following three properties hold:

1. **No poisoning:** For all i with $W_m^N[k, i] \neq 0$, $\tilde{\varphi}_{m-1,i}^N \neq \text{NaN}$. By the inductive hypothesis, $\tilde{\varphi}_{m-1,i}^N = \varphi_{m-1,i}^N$ for all such i .
2. **No divergence:** For all $i \in R_{m,k}(\varphi_{m-1}^{\text{pre}})$, $\tilde{\varphi}_{m-1,i}^N = \varphi_{m-1,i}^{\text{pre}}$. Combined with property 1, this gives $\varphi_{m-1,i}^N = \varphi_{m-1,i}^{\text{pre}}$ for all relevant inputs.
3. **No sign flips:** For all $i \in R_{m,k}(\tilde{\varphi}_{m-1}^N)$, $\text{sgn}(W_m^N[k, i]) = \text{sgn}(W_m^{\text{pre}}[k, i])$.

From properties 1 and 2, the input dependency vectors agree on all relevant inputs: $\varphi_{m-1,i}^N = \varphi_{m-1,i}^{\text{pre}}$ for all $i \in R_{m,k}(\varphi_{m-1}^{\text{pre}})$. Together with property 2, the relevant input sets are identical: $R_{m,k}(\varphi_{m-1}^N) = R_{m,k}(\varphi_{m-1}^{\text{pre}})$.

From property 3 and Lemma 3, the dependency propagation (2) yields the same result for W_m^N and W_m^{pre} on the shared input dependency vector. Therefore $\varphi_{m,k}^N = \varphi_{m,k}^{\text{pre}} = \tilde{\varphi}_{m,k}^N$. $\square$

3.2 Soundness of the Transfer Margin

Theorem 5 (Soundness of δ_{diff} with reused dependencies). *Let δ_{diff}^* be the true transfer margin (Definition 1 in the BRIDGE paper), and let $\hat{\delta}_{\text{diff}}$ be the value returned by the transfer MIP using the inferred dependencies $\tilde{\varphi}^N$ from Algorithm 1 instead of the full dependency analysis. Then $\hat{\delta}_{\text{diff}} \geq \delta_{\text{diff}}^*$.*

Proof. The transfer MIP computes an upper bound on δ_{diff}^* : $\hat{\delta}_{\text{diff}} \geq \delta_{\text{diff}}^*$. This holds because the MIP is a relaxation — its feasible region contains all true feasible points (x, x') satisfying the constraints of Definition 1.

Dependency constraints are *valid inequalities*: they hold for every feasible point but are not part of the problem definition. Specifically:

- By Theorem 4, every constraint added by Algorithm 1 (where $\tilde{\varphi}_{m,k}^N \in \{0, +1, -1\}$) is a correct ordering constraint for N . Adding a correct constraint does not remove any true feasible point, so the MIP remains a valid relaxation.
- For neurons where $\tilde{\varphi}_{m,k}^N = \text{NaN}$, no constraint is added. This only *enlarges* the feasible region compared to adding the (unknown) true constraint, which can only *increase* the objective value.

Therefore the feasible region of the MIP with reused dependencies *contains* the feasible region of the MIP with full dependencies, which in turn contains all true feasible points. The objective value $\delta_{\text{diff}}^{\hat{}}$ is thus a valid upper bound: $\delta_{\text{diff}}^{\hat{}} \geq \delta_{\text{diff}}^*$.

By Theorem 1 of the **BRIDGE** paper, $\delta_{\text{pre}}^* + \delta_{\text{diff}}^* \leq \delta^*$, and since $\delta_{\text{diff}}^{\hat{}} \geq \delta_{\text{diff}}^*$, we have $\delta_{\text{pre}}^* + \delta_{\text{diff}}^{\hat{}} \geq \delta_{\text{pre}}^* + \delta_{\text{diff}}^*$. The transfer property remains sound: any input certified robust using $\delta_{\text{diff}}^{\hat{}}$ (i.e., $\mathcal{C}(N, x, c) - \mathcal{C}(N_{\text{pre}}, x, c) > \delta_{\text{diff}}^{\hat{}}$) is truly robust, since $\delta_{\text{diff}}^{\hat{}} \geq \delta_{\text{diff}}^*$ means the certification condition is strictly stronger. $\square$

Remark 6 (Relationship to the inference theorem). The inference condition from the **BRIDGE** paper (Theorem 2) states: if $\mathcal{C}(N, x, c) - \mathcal{C}(N_{\text{pre}}, x, c) > \delta_{\text{diff}}$, then N is robust on x . When using $\delta_{\text{diff}}^{\hat{}} \geq \delta_{\text{diff}}^*$ from the reused-dependency MIP, the condition $\mathcal{C}(N, x, c) - \mathcal{C}(N_{\text{pre}}, x, c) > \delta_{\text{diff}}^{\hat{}}$ is *more conservative* (harder to satisfy) than using δ_{diff}^* . Therefore, every input certified by the reused-dependency analysis is also certified by the full analysis. Soundness is preserved; only the certification rate may decrease (fewer inputs may satisfy the stricter condition).

4 Complexity Analysis

Standard dependency analysis. For each layer m with k_m neurons and k_{m-1} inputs, the sign propagation costs $O(k_m \cdot k_{m-1})$ (matrix operations). For each neuron with $\varphi_{m,k} = \text{NaN}$, an LP sub-problem is solved, costing $O(\text{LP}(m))$ where $\text{LP}(m)$ depends on the MIP size up to layer m . In the worst case, all neurons are NaN, giving total cost $O(\sum_m k_m \cdot (k_{m-1} + \text{LP}(m)))$.

Transfer dependency reuse (Algorithm 1). The sign-flip set F_m is computed in $O(k_m \cdot k_{m-1})$ via element-wise sign comparison. The three checks per neuron (poisoned, diverged, flipped) each require $O(k_{m-1})$ in the worst case. Total cost: $O(\sum_m k_m \cdot k_{m-1})$ — the same as the sign propagation alone, with **zero LP sub-problems**.

Expected reuse rate. For fine-tuned networks where $\|\Delta W_m\|$ is small relative to $\|W_m^{\text{pre}}\|$, sign flips are rare (they require $|\Delta W_m[k, i]| > |W_m^{\text{pre}}[k, i]|$). At layer 1, the input dependencies are identical and few weights flip sign, so nearly all dependencies are reused. At deeper layers, the cascade effect from NaN neurons at earlier layers reduces the reuse rate, but for typical fine-tuning scenarios the degradation is gradual.

5 Discussion

What is preserved. Soundness and correctness of δ_{diff} as an upper bound on the true transfer margin. Every reused constraint is proven correct (Theorem 4). The transfer property and inference guarantees of **BRIDGE** are fully preserved (Theorem 5).

What may change. The tightness of the upper bound $\delta_{\text{diff}}^{\hat{}}$. Neurons where Algorithm 1 conservatively outputs NaN may have had a non-NaN dependency under the full analysis (either from sign propagation or from the LP sub-problems). The absence of these constraints enlarges the feasible region, potentially increasing $\delta_{\text{diff}}^{\hat{}}$.

When to use. This optimization is most effective when:

- ΔW is sparse or small (fine-tuning, pruning, noise injection),
- The standard-mode dependencies φ^{pre} are already computed,
- The LP sub-problems in the full analysis constitute a significant fraction of the dependency analysis runtime.

Possible extension. For neurons where Algorithm 1 sets $\tilde{\varphi}_{m,k}^N = \text{NaN}$ due to a sign flip, one could optionally *recompute* the dependency via the standard propagation rule (1) using W_m^N and $\tilde{\varphi}_{m-1}^N$, without solving an LP. This recovers some dependencies at the cost of a per-neuron matrix-vector product, still avoiding LP sub-problems. This creates a three-tier strategy: (1) reuse from N_{pre} when safe, (2) recompute analytically via sign propagation when the input dependencies are known, (3) set to **NaN** otherwise.